Fraunhofer AISEC

Fraunhofer Institute for Applied and Integrated Security AISEC

Alpine Linux Persistence and Storage Summit 2025

Detecting Supply Chain Attacks from the Filesystem Level with eBPF, fanotify and others

Tobias Specht
01.10.2025

Agenda

- **Background**
- Supply chain attacks and the XZ backdoor
- Previous work
- **Linux Tracing APIs**
- **Requirements**
- **API Experience**
- **Solution**
- **POC**
- **Future work**

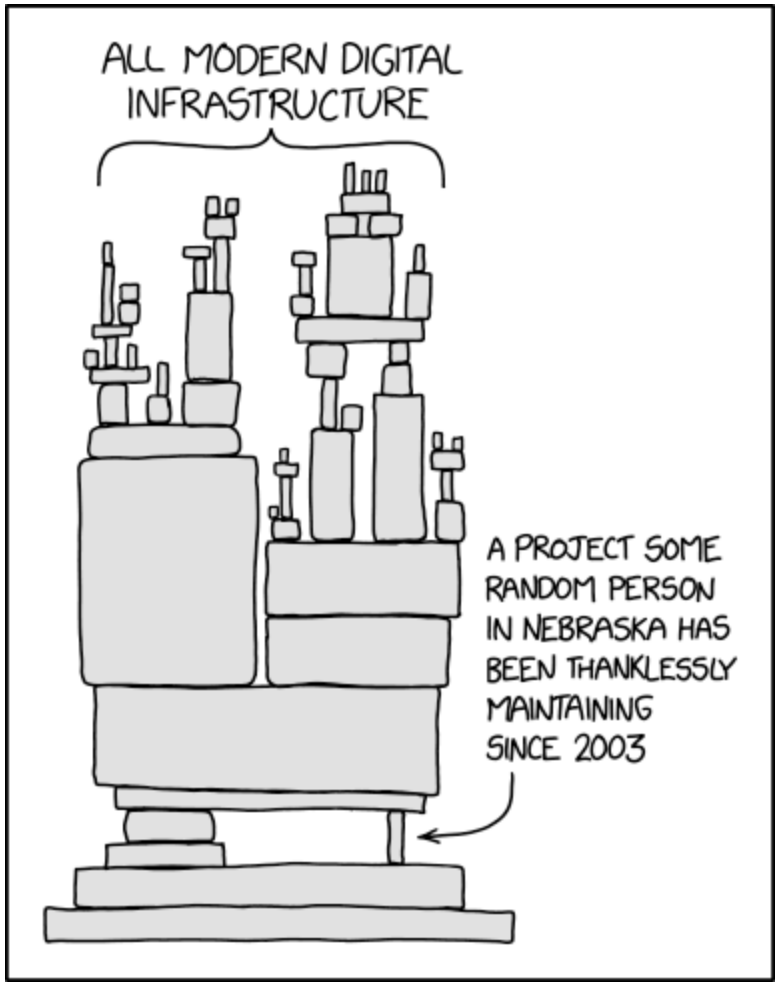
Whoami

Tobias Specht

- **Cybersecurity researcher based in Germany**
- **Working at the Fraunhofer Institute for Applied and Integrated Security (AISEC)**
- **Research focus on**
- Static Code Analysis
- Offensive Security
- Automotive and Embedded Domain
- **https://github.com/peckto**
- **Matrix: @peckto:tchncs.de**

Background

Background
Supply Chain Attacks

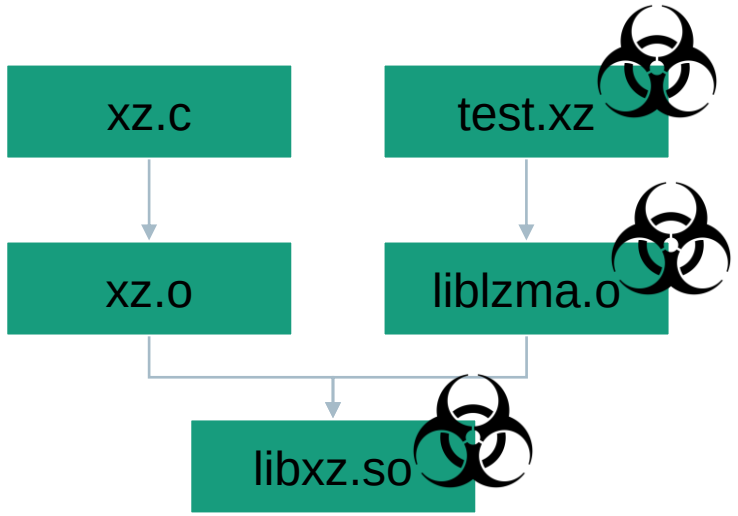


https://xkcd.com/2347

Background

XZ Backdoor CVE-2024-3094 (v5.6.1)

- Target of the attack was the openssh server, to gain remote code execution
- Infiltrating the indirect dependency xz
- Attack on xz was only detected by coincidence, it’s unknown how many undetected attacks exist (or have existed)
- Supply chain attack on xz was utilizing the build system to hide and inject its malicious code



Supply Graph
FOSDEM 2025

Method

- Trace execv to capture all program invocations (inspired by compile_commands.json)
- Reconstruct dataflow based on cli arguments (e.g. gcc source.c –o exe, cp source dest, …)

Limitations

- Need to parse cli
- Not all dataflows are visible (e.g. IPC)

Idea:

1. Monitor on filesystem level
2. Somehow trace pipes to capture relevant IPC

Supply Graph

FOSDEM 2025

README Apache-2.0 license

Analyze build process

In the docker container, the following Debian packet builds are included:

- xz-5.6.1 (CVE-2024-3094)
- xz-5.6.2
- openssh-9.2p1
- openssl-3.0.15

Run the analysis:

```
docker run --rm -it supply-graph
analyze-build-graph xz-5.6.1
```

Identified anomalies in the supply graph are displayed at the end of the log:

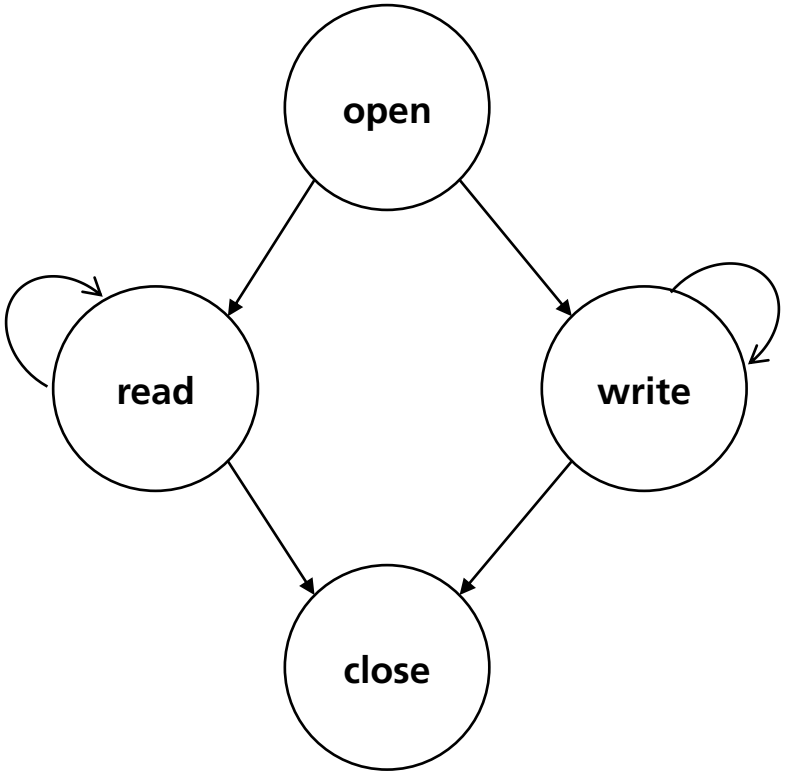
```
[...]
Root files not part of upstream:
* /data/xz-5.6.1/xz-utils-5.6.1/debian/normal-build/src/liblzma/liblzma_la-crc64-fast.o
Binary files without corresponding source code:
* /data/xz-5.6.1/xz-utils-5.6.1/debian/normal-build/src/liblzma/liblzma_la-crc64-fast.o
```

https://github.com/Fraunhofer-AISEC/supply-graph

Tracing on filesystem level

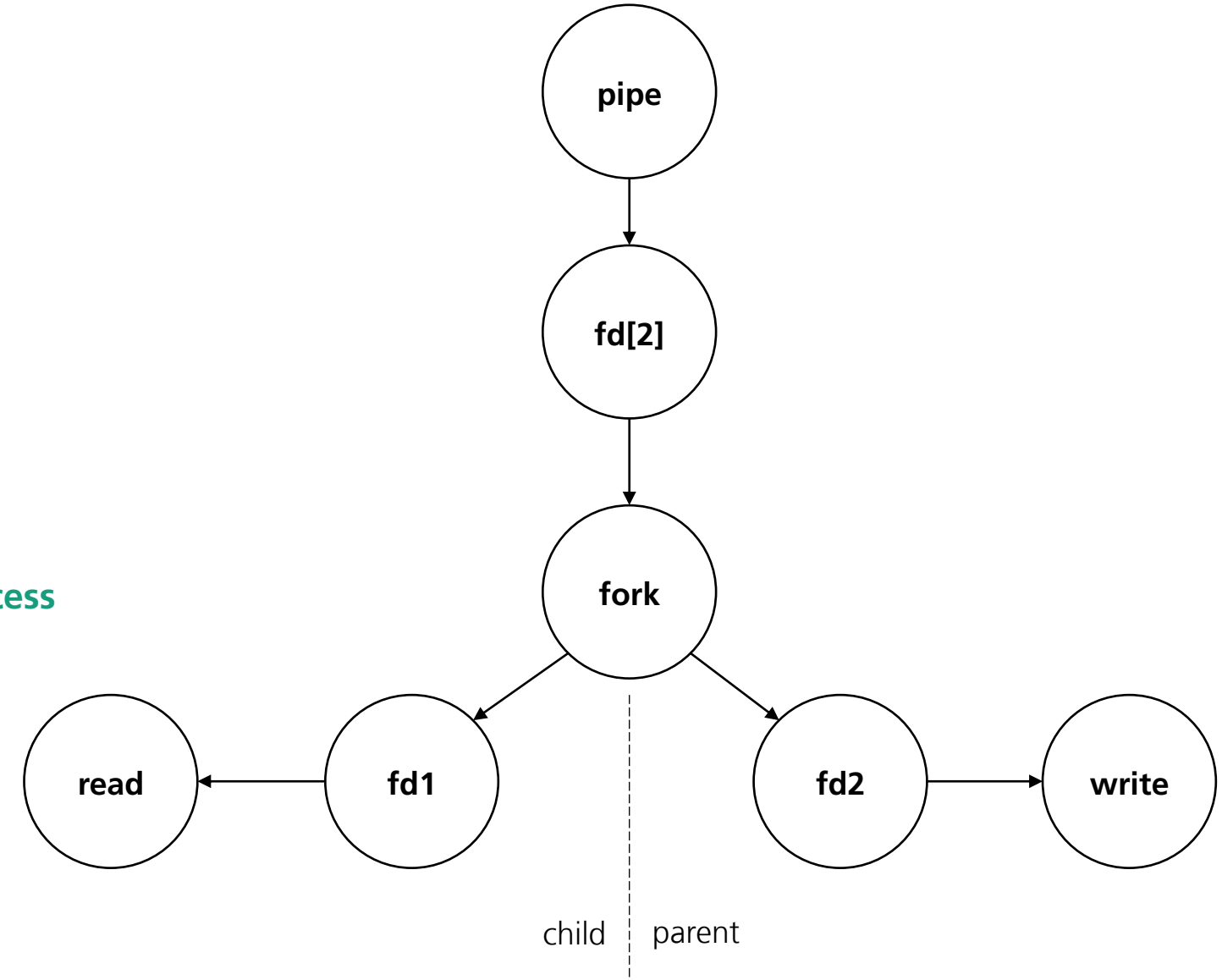
Tracking file access

- Operations: open, read, write, close
- File descriptor is not stateless



Tracking pipes

- **Operations**
- File: pipe, read, write, close
- Process: fork | clone
- **File descriptor is not stateless**
- **Need to track source and sink process**



Linux Tracing APIs

API	Space	Interface	Sync/Async
ptrace	User	Syscals	Sync
LD_PRELOAD	User	Library calls	Sync
eBPF	Kernel	Syscals	Async
Inotify	User	File operations	Async
fanotify	Root	File operations	Sync

Requirements

Requirement	ptrace	LD_PRELOAD	eBPF	Inotify	fanotify
Transparent	x	x	x	x	x
Stealth	-	-	x	x	x
Track file OP	x	x	x	x	x
Track pipe OP	x	x	x	-	-
Container	x	x	~	x	x
Complexity	low	medium	medium	medium	medium

API Experience

API Experience
ptrace (strace)

```
[pid 36398] openat(AT_FDCWD, "/lib64/liblzma.so.5", O_RDONLY|O_CLOEXEC) = 3
[pid 36399] openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
[pid 36399] close(3)                    = 0
[pid 36399] openat(AT_FDCWD, "/lib64/libtinfo.so.6", O_RDONLY|O_CLOEXEC <unfinished ...>
[pid 36398] close(3 <unfinished ...>
[pid 36399] <... openat resumed>)       = 3
[pid 36398] <... close resumed>)        = 0
```

- **Pro**
- Ready to run tool strace, with filter options
- Capture only events from target process(es)

- **Con**
- Output not machine readable (no json format)
- Difficult to align operations (unfinished…, resumed)
- Hooks would need custom implementation

API Experience
LD_PRELOAD

- **Pro**
- Full control
- Example projects to build up on
- Capture only events from target process(es)
- **Con**
- Not verry elegant
- Much manual effort
- Living inside the target process

API Experience
eBPF (bpftrace)

- **Pro**
- Transparent and stealth for the process
- Very performant
- High level language to describe trace points (bpftrace)
- **Con**
- Captures all system events! Need to filter wisely
- Async event queue and no guarantee for processing order (close might be processed bevor open)
- No function hooking or influence on the user process
- No access to userland (e.g. no access to file content, only to kernel structs)

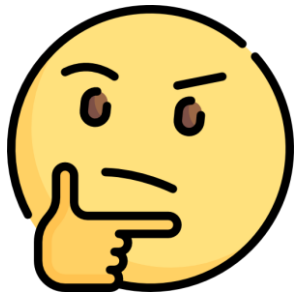
API Experience
fanotify

- **Pro**
- Permission events are sync
- Access to file content

- **Con**
- Captures all system events! Need to filter wisely
- Not all file events are sync (FAN_ACCESS_PERM, FAN_OPEN_PERM, FAN_OPEN_EXEC_PERM)
- No alignment between pre and post events

API Experience
Summary

- **ptrace** ❌
- **LD_PRELOAD** ❌
- **eBPF** 〰
- **fanotify** 〰



Open problems:

1. Stateless way to track pipes
2. Trace all file access in sync and capture content, before and after write

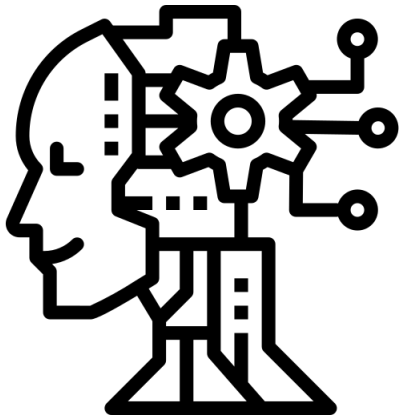
Solution

Solution
Part 1 – pipe

Me

This is my problem

- Use fstat(fd) and take st_ino
  That uniquely identifies the underlying pipe object while it exists
- Or read /proc/fd/ (a symlink like "pipe:[12345]");
  the number is the pipe's inode



AI

Solution

Part 2 – file access

Me

This is my problem

Sounds like you need to write your own filesystem

You can use FUSE fs, they have examples

Colleague

Solution

- **FUSE fs**
- Build up on libfuse passthrough example
- Convert fd to unique counter
- Track file operations
- Hash file content
- **Bpftrace**
- Track pipes based on inode number

Setup

- **VM with Debian 12**
- **FUSE mount / as /mnt**
- **Bind mound special file systems (e.g. /dev, /proc, …) on /mnt**
- **Chroot /mnt**
- bpftrace <trace_pipe.bpftrace> -c <target command>
- **Cleanup mounts**
- **Create data flow graph in KuzuDB**
- **Write graph queries in Cypher**

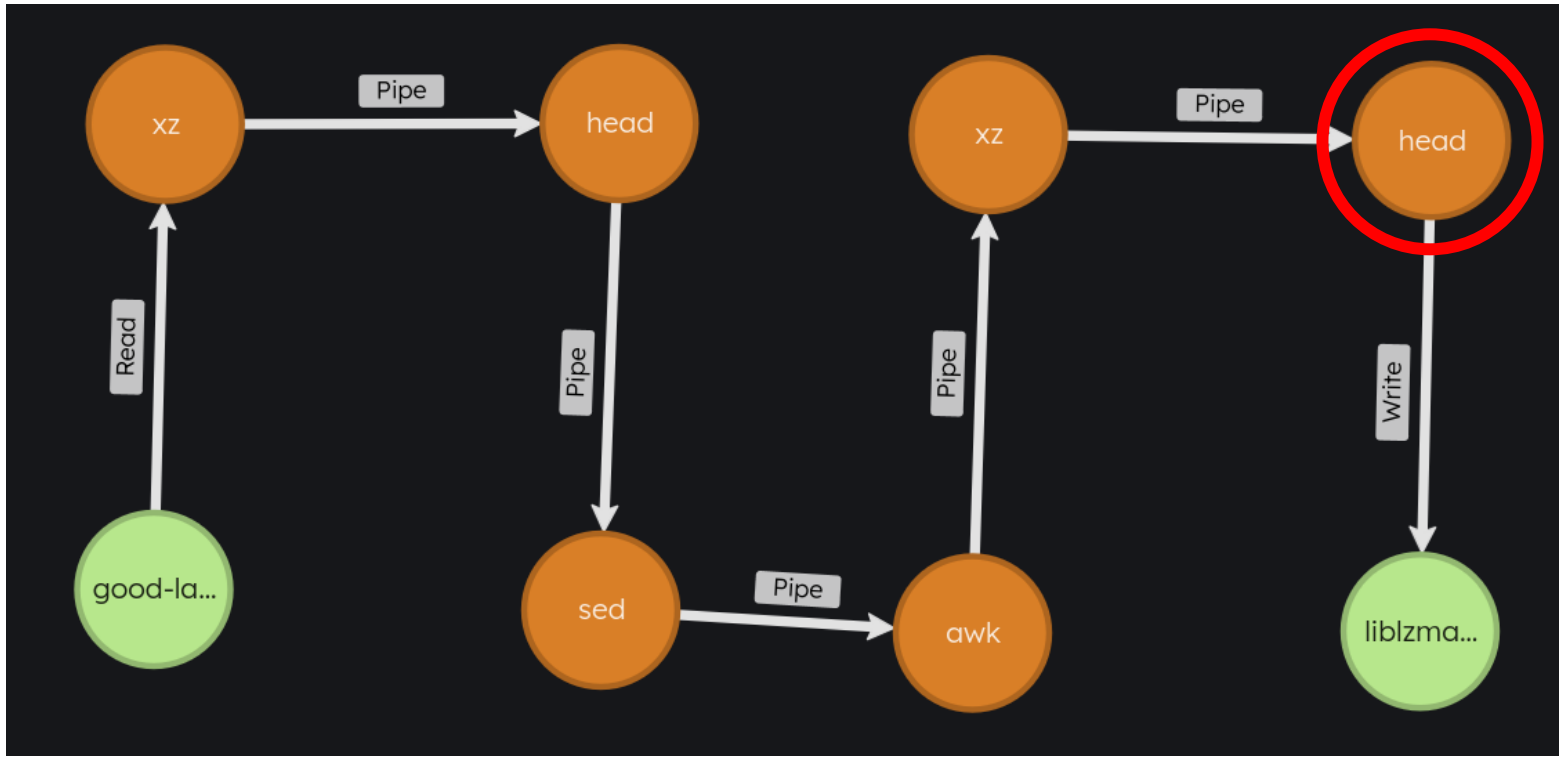
Limitations

- Currently no Docker support
- No support for other IPC methods (e.g. socket, queue, …)

POC

CVE-2024-3094

```
match (l:File {name: "good-large_compressed.lzma"})-[a* SHORTEST 1..30]->(f:File {path: "liblzma_la-crc64-fast.o"})
return l, a, f
```



POC

CVE-2024-3094

```
MATCH (p:Process)-[w:Write]->(f:File {type: ".o"})
where
p.name <> "strip" and
p.name <> "cp" and
p.name <> "as" and
p.name <> "ld" and
p.name <> "x86_64-linux-gnu-as" and
p.name <> "x86_64-linux-gnu-ld.gold"
RETURN f, p
```

Future Work

- Write more queries to detect anomalies
- Evaluate approach with more packages
- Integrate into package build CI

Thanks

- @rumpelsepp
- @dxld

Acknowledgements

This work was funded by the German Federal Ministry of Education and Research (BMBF) as part of the ALPAKA project:

https://www.forschung-it-sicherheit-kommunikationssysteme.de/projekte/alpaka

Contact

Tobias Specht
Group Embedded Software Analysis
Department Product Protection & Industrial Security
Phone +49 89 32299 86 187
tobias.specht@aisec.fraunhofer.de

Fraunhofer Institute for Applied and Integrated Security AISEC
Lichtenbergstr. 11
85748 Garching
Germany

Fraunhofer AISEC

Fraunhofer Institute for Applied and Integrated Security AISEC

https://www.cybersecurity.blog.aisec.fraunhofer.de